One-dimensional Cutting Stock with a Limited Number of Open Stacks: Bounds and Solutions from a New Integer Linear Programming Model

Claudio Arbib

Dipartimento di Ingegneria/Scienze dell'Informazione e Matematica

Università degli Studi dell'Aquila

via Vetoio, Coppito, I-67010 L'Aquila, Italy

e-mail: claudio.arbib@univaq.it

Fabrizio Marinelli

Dipartimento di Ingegneria dell'Informazione

Università Politecnica delle Marche

via Brecce Bianche 12, I-60121 Ancona, Italy

e-mail: fabrizio.marinelli@univpm.it

Paolo Ventura

Istituto di Analisi dei Sistemi e Informatica

Consiglio Nazionale delle Ricerche

viale Manzoni 30, I-00185 Roma, Italy

e-mail: paolo.ventura@iasi.cnr.it

Abstract

We address a 1-dimensional cutting stock problem where, in addition to trim-loss minimization, cutting patterns must be sequenced so that no more than s different part types are in production at any time. We propose a new integer linear programming formulation whose constraints grow quadratically with the number of distinct part types and whose linear relaxation can be solved by a standard column generation procedure. The formulation allowed us to solve problems with 20 part types for which an optimal solution was unknown.

1 Introduction

1.1 The problem

The 1-dimensional *Cutting Stock Problem* (1-CSP, see [8] for an old but vast annotated bibliography) calls for cutting d_i parts w_i units wide ($i \in B$, $|B| = m$) from a minimum number of identical stock items of width w. A *cutting pattern* specifies a way to cut parts from a stock item: in the 1-dimensional case, it corresponds to an integer solution $\mathbf{p} = (p_1, \ldots, p_m) \geq 0$ of inequality $\sum_{i \in B} w_i p_i \leq w$ (knapsack condition). A solution of the 1-CSP provides (i) a set P of distinct patterns, (ii) the number of times each pattern of P must be repeated in order to cover the parts requirement: this number is called the pattern *run length*. In practice, to reduce setups it is customary to non-preemptively run each pattern up to its run length. Together with P, the sequence in which patterns are used — in fact, a permutation π of P — gives a *cutting plan* (P, π).

Along with the classical objective of trim-loss minimization, a major goal of industries deploying a cutting process is to organize production in a way compliant with machine-tool resources and capacity, see e.g. [2, 3, 13]. Under this respect, an important indicator is the number of *open stacks*: after cut, parts of the same size are normally collected into a stack. A stack is open (closed) as soon as the first (the

last) of its parts has been cut. Because automated cutting machines usually have a downstream buffer with limited capacity, only a limited number s of stacks can be maintained open throughout the cutting process, and a problem arises of finding a feasible cutting plan.

In the literature, contributions can be found on the *Open Stack Minimization Problem*. This problem, known with the acronym MOSP, calls for permuting a given set of patterns so as to minimize the maximum number of open stacks. The MOSP was the subject of an international contest: in [15], Smith and Gent collected a quite large number of papers with algorithm details and computation experiments. Recent research has focussed on 0-1 matrices with the *Consecutive Ones Property* (C1P), providing an interesting integer linear programming formulation that uses Oswald-Reinelt inequalities [7]; but a breakthrough [6] has recently been achieved by a smart combinatorial lower bound that, used in a branch-and-bound scheme, reduced by a factor of 10^{-6} the best computation times in [15].

Apparently, the MOSP is to be solved within a decomposition approach where one first finds a set of patterns minimizing trim-loss, and then sequences those patterns minimizing the maximum number of open stacks. Unfortunately, this decomposition has a couple of important drawbacks:

- First, the problem is not the same as minimizing open stacks under a prescribed trim-loss: in fact, cutting plans with similar trim-losses may be obtained by many different pattern sets, and the quality of a pattern sequence (as well as the time needed to compute it) can strongly depend on the number and type of patterns chosen.

- Moreover, even an optimal solution to the latter problem may require more open stacks than allowed by the cutting machine, and therefore be infeasible in practice.

The correct way to address the constraints on buffers is, in fact, to find a pattern set that both covers demand and can be sequenced without exceeding buffer capacity: doing that with the goal of minimizing trim-loss is what we here call 1-*dimensional Cutting Stock with a Limited Number of Open Stacks Problem* (1-CS-LOSP).

To the best of our knowledge, the 1-CS-LOSP has so far registered few contributions. Belov and Scheithauer [4] propose an effective sequential heuristic, that makes use of pseudo-prices in order to select the part types to be cut pattern after pattern. Arbib et al. [1] design a tabu-search heuristic that explores the space of sequences in which part types are produced and, for each sequence considered, finds a pattern set by solving a constrained version of Gilmore-Gomory's model [9, 10]. Yanasse and Pinto Lamosa [17] formulate the problem as integer linear programming: but the formulation, with exponentially many variables and constraints, just helps develop a lower bound (by lagrangian relaxation) and a primal heuristic.

1.2 This contribution

In this paper we propose a new integer linear programming formulation of 1-CS-LOSP which, unlike Yanasse and Pinto Lamosa's, is compact in the number of constraints.

We start (Section 2) from the well-known Kantorovich's model for the 1-CSP [11] that describes how to cut each stock item via assignment variables. We then add 0-1 flow variables in order to count (and bound) the number of open stacks. These variables describe matrices with m rows called *tracks*. Each column of a track is the incidence vector of a set of part types to be produced in consecutive time intervals. Columns are organized in such a way that no more than s stacks can ever be open. The resulting model has a pseudo-polynomial number of variables and activation constraints.

Successively (Section 3), we convexify the knapsack constraints of the modi-

fied Kantorovich's model and obtain a Gilmore-Gomory model with track variables. While track variables remain the same, convexification transforms assignment variables into run lengths of cutting patterns. Because in the 1-CSP cutting patterns correspond to the solutions of an integer knapsack problem, the number of these variables is exponential in m. Constraints, however, are reduced and, due to the special structure of track matrices, become $O(m^2)$.

Let us briefly collocate our contribution in the research landscape, with special reference to [1, 4, 17].

Tracks are implicitly used in [1] within a tabu search algorithm. Every step of the search corresponds to a specific part type sequence β. The 1-CSP solved at β provides a feasible solution that uses patterns producing part types at reciprocal distance $\leq s-1$ in β. This condition corresponds, in fact, to generate patterns that are supported by the columns of a track. The track chosen has a nice effect on order spread, but its choice limits the search to a proper subset of the track space. Therefore, the algorithm in [1] is heuristic under two respects: first, because it is a local search on part type sequences; secondly, because even exploring all sequences does not cover the whole track space. In any case, [1] does not address at all the notion of track, nor track properties. As a matter of fact, an exhaustive search of the track space is not conceivable, even for toy instances, without exploiting the properties established in Section 4 of the present contribution. These properties also open a way to symmetry breaking: an issue still to be developed and out of the scope of the present paper, but briefly addressed in Section 4.2.

On the experimental side of the matter, a first result of our study is a way to find optimal solutions to medium-size instances (for instance, 20 part-type instances tested in [4] for which an optimum was not known). Such a result was out of reach of previous models. For larger instances, our study suggests two directions of improvement: lower bounds, and a deeper insight in the combinatorial nature of the

problem.

Good bounds are one of the crucial issues to find optimal solutions of difficult problems, and the 1-CS-LOSP is no exception. The very complex method developed in [17] generates, via Lagrangian relaxation, two separable problems. A drawback of the bound so obtained is that the relaxation has exponentially many multipliers, and this is well known to have bad effects on convergence (a deeper discussion on this method is developed in Section 2). A by far easier and stabler way of computing a lower bound is by solving a 1-CSP where patterns are not allowed to produce $> s$ part types (CSP(s)). CSP(s) generally provides good bounds; however, there is a difficulty in obtaining a primal solution from a fractional optimum, because the solution of the linear relaxation of CSP(s) has no evident relation with tracks: therefore it is not clear how to modify it so as to schedule its patterns appropriately. As far as our approach is concerned, a straightforward linear relaxation of the model developed in Section 3 is proved to be no better, in general, than that of CSP(s). However it is not difficult to strengthen it in several ways, as described in Section 4: thanks to these techniques we were able to certify optimality of some instances without solving any integer program (Section 5).

2 Formulating the problem

Let us formally state the 1-CS-LOSP as follows:

Problem 2.1 *Given*

- *an unlimited number of identical stock items of width w,*
- *a set of part types B of distinct widths w_i, required in amounts d_i, $i \in B$,*
- *a positive integer s,*

find cutting plan (P, π) *with run lengths that exactly cover the demand* $\mathbf{d} = (d_i)_{i \in B}$ *and such that*

- *the number of open stacks never exceeds* s*,*
- *the total number of stock items used (= sum of run lengths) is minimized.*

Non-identical stock items can easily be taken in consideration (see below) as well as a non-exact coverage of demand.

Yanasse and Pinto Lamosa [17] formulate Problem 2.1 by merging Gilmore-Gomory's model and a variant of Tang-Denardo's tool-switch formulation [16]. The resulting model:

- explicitly generates the patterns, the 0-1 incidence vectors of the part types produced by each pattern, and integer-valued run lengths y_k;
- uses assignment variables x_{kn} stating whether pattern k is or not sequenced in the n-th position; 0-1 variables u_{in} stating whether or not the n-th pattern feeds the i-th stack with a part type; 0-1 variables v_{in} stating whether the i-th stack is or is not still open between the n-th and the $(n+1)$-st pattern;
- links run lengths and assignment variables via 0-1 activation variables v_k that take value 1 if and only if $y_k > 0$.

Apparently, the formulation is independent on the dimension of parts, but complexity of course increases in the 2-dimensional case. Candidate patterns can grow exponentially with m, and therefore so does the number of both variables and constraints. In detail, the Authors count $p^2+2pm+2p-m$ variables and $p^2m+pm+5p+1$ constraints for p patterns, and observe that these numbers are quite large even for small values of m: for instance, $m = 20$ and $p = 1000$ already give $1,041,980$ variables and $20,025,001$ constraints, a number that can prevent from using the formulation even for small problems. Dual bounds can be obtained by a Lagrangian

decomposition of the model, but the very large number of multipliers (as many as the number of cutting patterns) affects the convergence speed and the numerical stability of the subgradient algorithm (a broad discussion on the topic can be found in [5] and [12]). Moreover the subgradient algorithm has to be accurately tuned and there is no guarantee that the procedure will converge in practice, nor that it is able to yield feasible solutions to the integrated problem. Finally, [17] is a model with *column-dependent-rows* (see [14]), thus its solution must recast to simultaneous column-and-row generation. In particular, the pricing problem must anticipate the values of the dual variables associated with the missing constraints: this can be done by the method proposed in [14] provided that a set of assumptions is satisfied. Summarizing, it is not clear both how to use and how effective is the method proposed in [17] to certify solution optimality.

In our formulation, we initially suppose that a stock item is cut at every time unit. To give a first formulation of Problem 2.1 we suppose to know an upper bound u to the number of stock items cut in an optimal solution. We also use an upper bound M_i to the number of parts of type i that can be cut from a stock item: in the 1-dimensional case with identical stock items a trivial M_i is $\lfloor \frac{w}{w_i} \rfloor$. Let us define the following decision variables for $i \in B$ and $j, l = 1, \ldots, u$:

- x_{il} = how many parts of type i are cut from the l-th stock item, $x_{il} \leq M_i$;
- $y_l = 1$ if the l-th stock item is at all cut, else 0;
- $q_{ij} = 1$ if the i-th stack is open at some time $\leq j$, else 0;
- $r_{ij} = 1$ if the i-th stack is closed at some time $\leq j$, else 0.

Variables x_{il}, y_l are the same as in Kantorovich's model [11]. Variables q_{ij}, r_{ij}, introduced to count open stacks, play a role similar to x_{in} in [17].

$$\min \sum_{l=1}^{u} y_l \tag{1}$$

$$\sum_{l=1}^{u} x_{il} = d_i \qquad i \in B \tag{2}$$

$$\sum_{i \in B} w_i x_{il} \leq w y_l \qquad i \in B, 1 \leq l \leq u \tag{3}$$

$$x_{il} \leq M_i(q_{ij} - r_{ij}) \qquad i \in B, 1 \leq l = j \leq u \tag{4}$$

$$q_{ij} \leq q_{i,j+1} \qquad i \in B, 1 \leq j < u \tag{5}$$

$$r_{ij} \leq r_{i,j+1} \qquad i \in B, 1 \leq j < u \tag{6}$$

$$\sum_{i \in B}(q_{ij} - r_{ij}) \leq s \qquad 1 \leq j \leq u \tag{7}$$

$$x_{il} \in \mathbb{N} \qquad i \in B, 1 \leq l \leq u \tag{8}$$

$$y_l, q_{ij}, r_{ij} \in \{0,1\} \qquad i \in B, 1 \leq j, l \leq u \tag{9}$$

Note that $x_{il} \geq 0$ and inequality (4) give $r_{ij} \leq q_{ij}$ for all i and j: that is, no stack can be closed if it has not been opened before. Non-identical stock items can be treated by using different capacities w in the right-hand side of knapsack constraint (3).

The upper bound u can be taken as large as the number of stock items cut in any feasible solution: trivially, using only patterns that produce a single part type, $u = \sum_{i \in B} \lceil \frac{d_i}{\lfloor w/w_i \rfloor} \rceil$ cuts are sufficient to cover demand opening one stack at a time.

The number of constraints involving q_{ij}, r_{ij} can be reduced by reducing the number of these variables. Indeed, one can group the cut sequence by a given number n of time intervals (an evaluation of n is provided in Theorem 3.4), and then refer q, r variables and inequalities (5)-(7) to the h-th interval instead of the l-th cut. With J_h denoting the set of cuts operated in interval h, inequality (4) is replaced by

$$\sum_{l \in J_h} x_{il} \leq d_i(q_{ih} - r_{ih}) \qquad i \in B, 1 \leq h \leq n \tag{10}$$

Although reduced, the number of variables and constraints of model (1)-(9) is still pseudo-polynomial and, as such, can be too large in practical cases. Another well known drawback of model (1)-(9) is that variable values can be exchanged without changing the solution, i.e., the model has a strong symmetry.

3 Reformulating the problem

Let $\mathbf{p}^k = (p_i^k)_{i=1}^m \geq 0$ denote a cutting pattern (i.e., a non-negative integer solution of $\sum_{i \in B} w_i x_i \leq w$), K be the index set of cutting patterns, $P = \{\mathbf{p}^k\}_{k \in K}$ be a set of patterns, and $\mathbf{T}$ a 0-1 matrix with $m = |B|$ rows, none of which are null. We call $\mathbf{T}$ a *track* for P if, for any $k \in K$, there exists a column h with $t_{ih} > 0$ for all $i \in B$ such that $p_i^k > 0$. Reciprocally, we say that P is *supported* by $\mathbf{T}$, and in particular that a single part type i is supported by a column $\mathbf{t}_h$ of $\mathbf{T}$ if $t_{ih} = 1$. With no loss of generality, from now on we assume that the columns of $\mathbf{T}$ are mutually non-dominated, that is, no two columns $\mathbf{t}_j, \mathbf{t}_h$ are such that $t_{ij} \leq t_{ih}$ for all $i \in B$.

Let $\omega(\mathbf{T})$ denote the largest number of non-zero elements in a column of a track $\mathbf{T}$. We then say that $\mathbf{T}$ is *feasible* if it has the *consecutive one property* (C1P) by rows, i.e., $t_{ij} = t_{ik} = 1 \Rightarrow t_{ih} = 1$ for $j < h < k$ and all $i \in B$, and $\omega(\mathbf{T}) \leq s$. From now on, suppose the columns of $\mathbf{T}$ are ordered according to the C1P by rows.

Proposition 3.1 *A set P of cutting patterns is schedulable if and only if it is supported by a feasible track $\mathbf{T}$.*

Proof. Suppose P schedulable in an order $\pi = (1, \ldots, n)$, and let $\bar{\mathbf{T}}$ be an $m \times n$ 0-1 matrix such that $\bar{t}_{ik} = 1$ if and only if the k-th pattern of the order produces parts of the i-th batch. Fill then each row of $\bar{\mathbf{T}}$ with the least number of 1's required to give it the C1P, remove dominated columns and call $\mathbf{T}$ the resulting matrix, which clearly supports P. Now every 0 replaced by a 1 corresponds to a part type not yet completed, i.e., to an open stack, and since π maintains open $\leq s$ stacks, every

column of $\mathbf{T}$ has no more than s non-zero elements: therefore, $\mathbf{T}$ is feasible.

Conversely, let $\mathbf{T}$ be a 0-1 matrix with the C1P and $\omega(\mathbf{T}) \leq s$, and let P be a set of patterns supported by $\mathbf{T}$, that is, for any pattern of P there exists a column h such that $t_{ih} = 1$ if part type i is produced by that pattern. Since the columns of $\mathbf{T}$ are mutually non-dominated, at least one part type supported by column h, say i, is not supported by column $h+1$ (and vice-versa). Since $\mathbf{T}$ has the C1P, once a part type is no longer supported by a column, it is not supported by any of the subsequent ones. So no pattern of P following those supported by the h-th column produces part type i, and since no more than s part types are supported by any column, P is schedulable. □

By Proposition 3.1, Problem 2.1 is restated without mentioning pattern sequencing:

Problem 3.1 *Find a set P of cutting patterns that produces all the parts required with a minimum trim loss and is supported by a feasible track.*

We now exploit the idea of track in a new formulation. Let $A = \{\mathbf{p}^1, \ldots, \mathbf{p}^{|K|}\} \subseteq \mathbb{N}^m$ denote the set of all feasible solutions to the knapsack condition

$$\sum w_i p_i^k \leq w \qquad \mathbf{p}^k \in \mathbb{N}^m$$

Using $\lambda_l^k \in \{0,1\}$ to select the k-th element of A, the pattern applied to the l-th stock item is rewritten

$$\mathbf{x}_l = \sum_{k=1}^{|K|} \mathbf{p}^k \lambda_l^k \tag{11}$$

with $\sum_k \lambda_l^k = 1, \lambda_l^k \geq 0$ and integer. Replacing x_{il} by (11) in (1)-(3), (9), and setting $z^k = \sum_l \lambda_l^k$ we get the Gilmore-Gomory's model

$$
\begin{aligned}
\min & \sum_{k=1}^{|K|} z^k & & & & (12)\\
& \sum_{k=1}^{|K|} p_i^k z^k & = & d_i & i \in B & \\
& z^k & \in & \mathbb{N} & k = 1, \ldots, |K| &
\end{aligned}
$$

which, substituting exponentially many variables for pseudo-polynomially many constraints of type (3), is well known to perform much better than Kantorovich's model both in terms of tightness of the linear relaxation, and because of the absence of symmetric solutions. Let $\mathbf{T}$ be an $m \times n$ feasible track, K_h contain the indices of patterns $\mathbf{p}^k$ supported by the h-th column of $\mathbf{T}$, and let z_h^k denote the run length of the k-th pattern of K_h. Plug $z^k = \sum_{h=1}^{n} z_h^k$ into (12), and note that variables q_{ih} and r_{ih} constrained as in (5), (6), (10) give $t_{ih} = q_{ih} - r_{ih}$. These variables can therefore be used to reformulate Problem 2.1:

$$
\begin{aligned}
\min & \sum_{h=1}^{n} \sum_{k \in K_h} z_h^k & & & & (13)\\
& \sum_{h=1}^{n} \sum_{k \in K_h} p_i^k z_h^k & = & d_i & i \in B & (14)\\
& \sum_{k \in K_h} p_i^k z_h^k & \leq & d_i(q_{ih} - r_{ih}) & i \in B, 1 \leq h \leq n & (15)\\
& \sum_{i=1}^{m} (q_{ih} - r_{ih}) & \leq & s & 1 \leq h \leq n & (16)\\
& q_{ih} - q_{i,h+1} & \leq & 0 & i \in B, 1 \leq h < n & (17)\\
& r_{ih} - r_{i,h+1} & \leq & 0 & i \in B, 1 \leq h < n & (18)\\
& z_h^k & \in & \mathbb{N} & 1 \leq h \leq n, 1 \leq k \leq |K| & (19)\\
& q_{ih}, r_{ih} & \in & \{0, 1\} & i \in B, 1 \leq h \leq n & (20)
\end{aligned}
$$

It is easy to see that model (13)-(20) derives from Dantzig-Wolfe decomposition

of integer program (1)-(3),(5)-(9),(10) rewritten with q_{ih}, r_{ih}.

Compactness of the new model depends on n. To evaluate it, it is useful to point out dominance properties among tracks.

Lemma 3.2 *Every feasible track* $\mathbf{T}$ *with* $\omega(\mathbf{T}) = \omega$ *is dominated by a feasible track with exactly* ω *non-zero elements per column.*

Proof. Let C be a subset of consecutive columns of $\mathbf{T}$ having $< \omega$ non-zero elements. Clearly, C does not consist of all the columns of $\mathbf{T}$, otherwise $\omega(\mathbf{T}) < \omega$. Thus, w.l.o.g., a column $\mathbf{t}$ with exactly ω non-zero elements follows the rightmost column, say $\mathbf{v}$, of C. Since by general assumption $\mathbf{v}$ and $\mathbf{t}$ do not dominate each other, there are at least two rows i, j such that $v_i < t_i$, $v_j < t_j$. In other words, as $\mathbf{T}$ has the C1P, t_i and t_j are the leftmost 1's of rows i and j, and therefore one can replace v_i by 1 preserving the C1P and mutual non-dominance, so augmenting the non-zeroes of $\mathbf{v}$. Repeating the argument one gets the thesis. □

Lemma 3.3 *Every feasible track* $\mathbf{T}$ *with* $\omega(\mathbf{T}) = s$ *is dominated by a feasible track where any two adjacent columns differ by exactly two elements.*

Proof. Suppose that consecutive columns $\mathbf{v}, \mathbf{t}$ differ by more than two elements: w.l.o.g. suppose $v_i > t_i$ for $i = 1, \ldots, h$ and $v_i < t_i$ for $i = h+1, \ldots, 2h$, for some $h > 1, h \leq s$. Then we can insert new columns $\mathbf{v}_j$ between $\mathbf{v} = \mathbf{v}_1$ and $\mathbf{t}$, $1 < j \leq h$, so that

$$\begin{cases} v_{ij} > v_{i,j+1} & \text{for } i = j \\ v_{ij} < v_{i,j+1} & \text{for } i = j + h \\ v_{ij} = v_{i,j+1} & \text{otherwise} \end{cases}$$

The resulting track has the C1P and supports any cutting plan supported by $\mathbf{T}$. □

We then conclude

Theorem 3.4 *The number of constraints* (14)-(20) *grows with* m^2.

Proof. In fact, Lemmas 3.2 and 3.3 imply that the non-dominated tracks have $n = m - s + 1$ columns. □

With $m = 20$ and $s = 4$ the number of constraints becomes $1,017$, dropping that in [17] by almost $20,000$ times. But, beside the size and the (possible) strength of the model, the main advantage of (13)-(20) is that its linear relaxation can be solved by a standard column generation procedure, overcoming in this way the numerical difficulties described in Section 2.

4 Computing a lower bound

Meaningful instances of 1-CS-LOSP usually have small values of s, because the smallest the s, the largest gap is expected between the optima of 1-CSP and 1-CS-LOSP. This circumstance can have a favorable implication on computation. In fact, for small s (e.g., $s = 2, 3$) the pattern set K does not increase with m as rapidly as in the 1-CSP. Consequently, K can be generated explicitly and, thanks to Theorem 3.4, non-trivial medium-size problems can often be solved without resorting to a pricing algorithm (see Section 5). For large instances, however, the availability of good dual bounds and primal solutions remains a crucial issue: in all our numerical tests we observed a fast convergence to a gap near 1, and most of the time consumed in an attempt to close it. Let $CSP(s)$ indicate the Gilmore-Gomory's model for 1-CSP where no pattern can produce more than s distinct types. $CSP(s)$ is not a formulation for 1-CS-LOSP but the solution of (the linear relaxation of) $CSP(s)$ provides a valid lower bound $z^R_{CSP(s)}$ to 1-CS-LOSP.

Proposition 4.1 *The linear relaxation of (13)-(20) provides no better bound than* $z^R_{CSP(s)}$.

Proof. Let $(z^1, \ldots, z^{|K|})$ be a (fractional) optimal solution of the linear relaxation of $CSP(s)$. For all $k \in K_h = K$ and all h, rewrite $z^k_h = x_k f$ choosing appropriate

$0 < f < 1$ and x_k; set also $q_{ih} = 1$ and $r_{ih} = (1 - f)$ for all i and h. This solution has value $z^R_{CSP(s)}$, and is feasible for the linear relaxation provided that $\frac{1}{n} \leq f \leq \frac{s}{m}$, the left inequality due to constraint (14) and the right one due to (16). But in fact $\frac{1}{n} = \frac{1}{m-s+1} \leq \frac{s}{m}$ if and only if $(s-1)(m-s) \geq 0$, which holds true for all $m \geq 0$ and $0 \leq s \leq m$. □

Despite Proposition 4.1, (13)-(20) can be used to get lower bounds tighter than those computed by $CSP(s)$, because its linear relaxation can be strengthened in several ways. For instance, a bound at least as tight as $z^R_{CSP(s)}$ can simply be obtained by writing model (13)-(20) upon CSP(s), i.e., by considering only patterns with no more than s distinct types (note that after relaxing variables, constraint (15) does not necessarily prohibit patterns with $> s$ distinct types). Also, constraint (15) can be replaced by

$$\sum_{k \in K_{ih}} z^k_h \leq z_U(q_{ih} - r_{ih}) \tag{21}$$

where $K_{ih} \subseteq K_h$ contains the patterns of K_h that produce part type i, and z_U is (an upper bound to) the value of an optimal solution of (13)-(20). The best choice depends on the ratio d_i/p^k_i and on z_U.

Finally, a better bound can be obtained by relaxing just constraint (19). To compute it, use Proposition 3.1 and Lemma 3.2 to enumerate non-dominated feasible tracks. Let $\mathbf{T}_k$ denote the submatrix consisting of the first k columns of a track $\mathbf{T}$ and $B_k = B - \{i \in B : t_{ik} = 0 \text{ and } t_{ij} = 1 \text{ for some } j < k\}$. Let also $z_{LB}(\mathbf{T}_k)$ be a lower bound to the optimum attainable with patterns supported by the uncompleted track $\mathbf{T}_k$. Enumeration can be performed by calling, for any 0-1 m-vector $\mathbf{t}$ with s non-zeroes, the following recursive function `track_enum()` with parameters 1, $\mathbf{t}$ and B:

`track_enum`(k, $\mathbf{T}_k$, B_k)

`if` $z_{LB} \geq z_{UB}$ `then end`

`For all` columns $\mathbf{t}$ with s non-zeroes in B_k and such that $\mathbf{t} \cdot \mathbf{t}_k = s - 1$ `do`

1) $\mathbf{T}_{k+1} := [\mathbf{T}_k|\mathbf{t}]$

2) `For` $i \in B_k$ `do if` $t_{ik} > t_i$ `then` $B_{k+1} := B_k - \{i\}$

3) `if` $z_{LB}(\mathbf{T}_{k+1}) \leq z_{LB}$ `then`

`if` $k = m - s$ `then` $z_{LB} = z_{LB}(\mathbf{T}_{k+1})$ `else track_enum`$(k+1, \mathbf{T}_k, B_k)$

where z_{UB} and z_{LB} are the best known integer solution and the best lower bound found so far, respectively. At the beginning of track enumeration, we set $z_{LB} = \lceil z^R_{CSP(s)} \rceil$ and z_{UB} equal to the solution found by *first-sequence-then-cut* heuristic proposed by [3]. Clearly, enumeration proceeds as far as $z_{LB} < z_{UB}$.

Removing row i from B_k at Step 2 corresponds to fixing to 0 the i-th element of any columns generated from then on: this ensures that the resulting matrix has the C1P. As soon as $m - s + 1$ elements have been fixed, non-dominated columns with s non-zeroes cannot be added any longer. Step 3 performs fathoming: call $\bar{K}$ the set of the cutting patterns that either (i) are supported by $\mathbf{T}_{k+1}$, or (ii) produce $\leq s$ part types in B_{k+1}. Then $z_{LB}(\mathbf{T}_{k+1})$ is the round-up of the optimum value of linear program (22), which can be computed by a standard column generation procedure.

$$z_{LB}(\mathbf{T}_{k+1}) = \left\lceil \min\{ \sum_{k \in \bar{K}} x_k | \sum_{k \in \bar{K}} p_i^k x_k = d_i, i \in B, x_k \geq 0, k \in \bar{K}\} \right\rceil \tag{22}$$

4.1 Size of search space

How much demanding is track enumeration? An upper bound to the number N of feasible tracks can easily be computed: the first column can be chosen in $\frac{m!}{s!(m-s)!}$ ways. The second column differs for one 1 turned into 0 and one 0 turned into 1: the former can be chosen in s ways, the latter in $m - s$. In the third column, the 1 changed to 0 cannot change anymore, so we have $s(m - s - 1)$ options left, and so

on. Summarizing,

$$N \leq \frac{m! s^{m-s}}{s!} = U_1$$

For $m = 10$ and $s = 3$ we get $N \leq 16,934,400,000$; for $m = 20$ and $s = 3$, $N \leq 3.19 \cdot 10^{23}$. This is just a bound because different tracks can formulate the same problem: for instance, with $s = 1$ there are $m!$ identity matrices, but any column-permutation yields the same optimum (see also Section 4.2). The following theorem gives another bound to N (and its exact value in closed form):

Theorem 4.2

$$N \leq \frac{s^m m!(m-s)!}{m^s [\Gamma(m/s)]^{2s}} = U_2 \tag{23}$$

where $\Gamma(x) = \int_0^\infty \tau^{x-1} e^{-\tau} d\tau$ *is Euler's Gamma function.*

Proof. A feasible non-dominated track $\mathbf{T}$ defines a set of transitions between states. Each state is denoted by the part types that, for some time, occupy the s buffers. The first column of $\mathbf{T}$ represents the initial state: when passing to the second column, a part type definitely leaves a buffer (and the system) and is replaced by a new one. A track counts $m - s + 1$ states, and tracks are as many as the different evolutions among these states. A single evolution counts $m - s$ transitions and is in a one-to-one correspondence with a pair $\langle \sigma, \mathcal{A} \rangle$, where σ is a sequence in which buffers are left by the part types, and $\mathcal{A} = \{A_1, \ldots, A_s\}$ is a partition assigning a non-empty subset of part types to each buffer. Note that permuting the elements in A_i gives the same track, so we just have to count partitions. Sequence σ is a word of length $m - s$ (the transitions) with characters taken from an alphabet with s symbols A, B, C, ..., and with the i-th symbol repeated $|A_i| - 1$ times. In conclusion, feasible non-dominated tracks are

$$N = \sum_{\mathbf{a} \in S} \frac{m!}{a_1! \ldots a_s!} \frac{(m-s)!}{(a_1 - 1)! \ldots (a_s - 1)!}$$

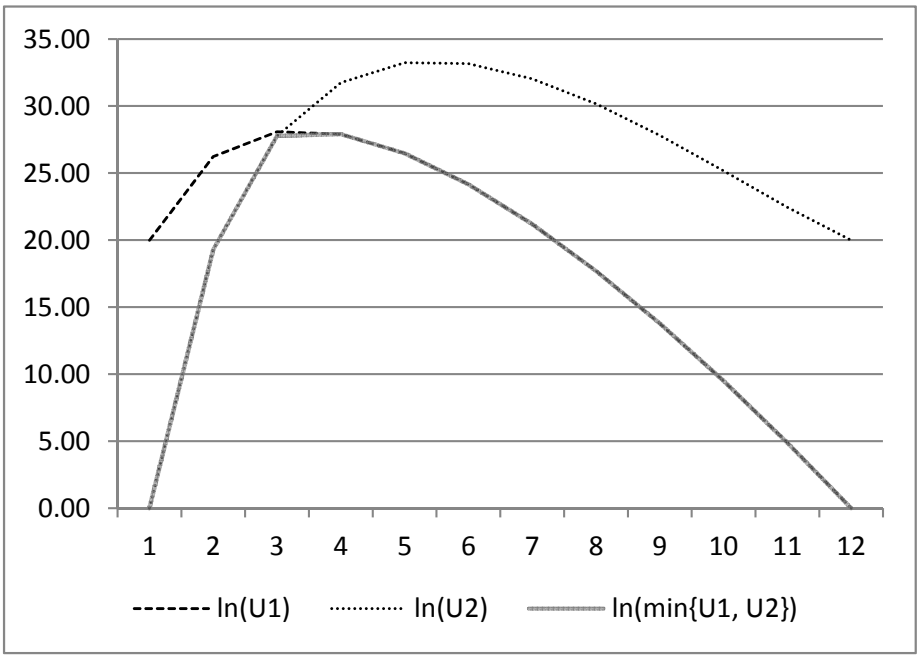


Figure 1: $\ln(\min\{U_1, U_2\})$ as a function of s.

where $a_i = |A_i|$ and $S = \{\mathbf{a} \in \mathbb{N}^s : a_1 + \ldots + a_s = m, a_i > 0\}$ (with $|S| = s^{m-s}$).

Since $\min\{\prod_{i=1}^{s} a_i! : a_1 + \ldots + a_s = m\}$ is achieved by a balanced partition, we can write

$$a_1! \ldots a_s! \geq [\Gamma(\frac{m}{s} + 1)]^s$$

We then get (23) recalling that $\Gamma(x+1) = x\Gamma(x)$. □

Figure 1 draws $\ln(\min\{U_1, U_2\})$ for $m = 12$ and $1 \leq s \leq 12$. The maximum, $1.31 \cdot 10^{12}$, is attained for $s = 4$.

4.2 Symmetry breaking

Symmetry breaking means here to identify unnecessary equivalent tracks in order to reduce the search space. Tracks corresponding to column permutations are equiv-

alent in the sense that produce identical optimal solutions and lower bounds by program (22). A maximal set of equivalent tracks is an *orbit* of a permutation group $\mathcal{G}$. Of course, we are not interested in all the column permutations of $\mathcal{G}$, but only in those, called feasible, which preserve the C1P: one is for instance the *mirror* permutation μ that sends each column $j \in C$ into $|C| - j$. Function `track_enum()` can easily be implemented so as to detect the orbits of $\mathcal{G}$: it is sufficient to introduce a lexicographical order among the columns of $\mathbf{T}$:

$$\mathbf{p} \prec \mathbf{q} \iff \sum_{i=1}^{m} 2^{p_i} < \sum_{i=1}^{m} 2^{q_i}$$

and always choose the least column $\mathbf{t}_{k+1}$ that differs from $\mathbf{t}_k$. To generate all the feasible orbits it is then sufficient to start the recursion with lexicographically increasing columns $\mathbf{t}_1$. However, this enumeration strategy may clash with other rules, such as fathoming, best-bound etc., adopted to direct the search. Reducing this conflict means limiting the use of lexicographical order to the extent just necessary, which can be done by investigating the form of feasible permutations. To this purpose, let us introduce the following notion.

Definition 4.1 *Let* $\mathbf{T}$ *be a track with columns indexed in* C*. Then* $S \subseteq C$ *is said to be* permutable *if any permutation* π *such that* $\pi(j) = j$ *for* $j \in C - S$ *is feasible.*

Maximal permutable sets are defined in the obvious way. We call trivial a permutable set consisting of a single column, and we say that a row $i \in B$ of a track $\mathbf{T}$ is a *unit row* if it contains exactly one non-zero element. In order to identify equivalent tracks we characterize permutable sets as follows.

Theorem 4.3 $S \subseteq C$ *is permutable if and only if for any row* $i \in B$ *either* $t_{ij} = t_{ik}$ *for all* $j, k \in S$ *or* i *is a unit row of* $\mathbf{T}$*.*

Proof. The 'if' part is obvious. Conversely, suppose that S hits non-identical entries of non-unit row i, say $q_{ij} \neq q_{ik}$, $j, k \in S$. Then any permutation that swaps j and k destroys the C1P. □

On the basis of Theorem 4.3 it can be observed that every non-trivial permutable set S_t hits at least $|S_t|$ unit rows, for otherwise C would have identical columns. Thus a permutable set is identified as soon as a unit row is detected during the generation of a track, i.e., when the current uncompleted track $\mathbf{T}_k$ contains a row i for which $t_{ik-2} = 0, t_{ik-1} = 1$ and $t_{ik} = 0$. Hence to avoid tracks having the same permutable set it is sufficient to apply the lexicographic order to the row-subsequences 010.

It is easy to verify that for $s \leq 3$ the permutable sets of a non-dominated feasible track are only the trivial ones.

As observed, one kind of feasible permutation is *mirroring*, i.e., reversing the order of the columns of a track. Mirroring can be detected as follows: read each column of a track $\mathbf{T}$ as a binary number and let $d_F(\mathbf{T})$ and $d_L(\mathbf{T})$ be the decimal encoding of the first and the last column of $\mathbf{T}$, respectively. Also, let $D_L(\mathbf{T}_k)$ be an upper bound to the decimal encoding of the last column of all the tracks achievable from the partial track $\mathbf{T}_k$. Notice that $D_L(\mathbf{T}_k)$ can easily be computed by setting to 1 the s most significant non-fixed rows of $\mathbf{T}_k$. If $D_L(\mathbf{T}_k) < d_F(\mathbf{T}_k)$ and tracks are enumerated by increasing d_F, then all the extension of $\mathbf{T}_k$ are tracks whose mirroring has been already generated.

5 Computational results

The method exposed was validated by a computational test on forty random instances with $m = 10$ part types and on the twenty instances with $m = 20$ part types reported in [4]. The test was run twice, with $s = 2$ and with $s = 3$: in all, we solved one hundred and twenty problems. Some tests with $s = 4$, not reported in the tables, confirmed that meaningful values of s are rather small (see also Section

4): indeed, in the $s = 4$ instances, we always found that the optimal solution has the same value as the standard unconstrained 1-CSP — with just one exception: problem 1 with 10 part-types, where the two values differ by 1.

Optimal solutions were computed by solving the integer program (13)-(20) with CPLEX 12.5. Lower bounds were computed by enumeration of tracks using the recursion `track_enum` described in Section 4. The algorithm was coded in C++ and compiled with Microsoft *cl* compiler (version 12.00.8804) with option /O2. Numerical precision was set to 10^{-6}. Linear programs were solved by LPSolve 5.5.0.9. Upper bounds z_{UB} were computed by the *first-sequence-then-cut* heuristic proposed by [1]. All the problems were solved on an Intel Core 2 Duo CPU E8500 3.16 GHz with 3Gb RAM.

Results are reported in Tables 1 (instances with 10 part types) and 2 (instances with 20 part types). Problems are identified by the first two columns, that respectively give the instance identifier and the value of s chosen (in Table 2, ID i stands for m20l100l7000L10000b1b100-i, see [4]). The third column reports the size of the complete set of maximal patterns. Columns 4 and 5 show the lower bounds z^R_{CSP} and $z^R_{CSP(s)}$. The last column gives the value z_{UB} of the best integer solution found in [1]. Let us separately comment the two tables.

Of the eighty problems with 10 part types, Table 1 just lists the thirty-three cases with $z_{UB} - z^R_{CSP(s)} \geq 1$. In the remaining forty-seven cases (nearly 59% of the test, 21 for $s = 2$ and 26 for $s = 3$) the linear relaxation of $CSP(s)$ gave a lower bound $\lceil z^R_{CSP(s)} \rceil$ identical to the best feasible solution z_{UB}, which therefore turned out to be optimal. Columns 6 to 8 detail the performance of `track_enum()`, reporting the obtained lower bound z_{LB}, the number of calls and CPU time; the next group of two columns details the performance (optimum value and CPU time) of model (13)-(20). Let us briefly comment these results.

- A first comment regards the gap between the optima of CS-LOSP and of CSP,

(the latter turned always out $\lceil z^R_{CSP} \rceil$). For $s = 2$ ($s = 3$), the gap found is 1.57% (0.28%) on average, 5.8% (2.3%) maximum; no gap was found in 8 (29) cases.

- The optimum of CSP(s) is computed in very short time (max 0.297 sec). We found $z_{CSP(s)} = \lceil z^R_{CSP(s)} \rceil$ in all instances but two (problems 9 and 14, with $s = 2$, where we found $z_{CSP(s)} = \lceil z^R_{CSP(s)} \rceil + 1$).

- In four cases (problem 14 with $s = 2$ and problems 1, 4, 37 with $s = 3$) track enumeration strictly improves $\lceil z^R_{CSP(s)} \rceil$; when $s = 3$, this permits to certify the optimality of the primal value z_{UB} found by tabu-search [1].

- When mirror tracks are not filtered (see Section 4.2), the calls to `track_enum()` increase in three cases only: problem 14 with $s = 2$ (from 2,120 to 3,134), problem 1 with $s = 3$ (from 12,823 to 17,024) and problem 4 with $s = 3$ (from 3,455 to 3,581). Note that these problems are among those requiring the largest CPU time in the test.

- CPLEX solved model (13)-(20) to optimality within 1,800 seconds in all cases but one (problem 28 with $s = 2$), reducing to zero a gap that in most cases consisted of just one stock item. The mean computation time is 141 seconds; however, sixteen of the 33 problems shown in Table 1 were solved in < 20 seconds.

- The mean number of calls to `track_enum()` (i.e., of nodes of the enumeration tree), the mean number of feasible patterns, and the average cpu time for computing z_{LB} are respectively 591, 142 and 0.37 seconds for $s = 2$ and 3849, 594 and 4.21 seconds for $s = 3$.

Table 2 lists the complete results for problems with 20 part types. Columns 6 to 8 give the best upper bound, the CPU time and the best lower bound obtained

by the CPLEX solution of (13)-(20). Values in bold denote optimality, certified in eighteen cases out of forty — it must be observed that in three of these problems with $s = 2$ and three with $s = 3$ the initial gap $z_{UB} - z^R_{CSP(s)}$ was 0. The time limit (3,600 sec) was reached in problem 1 only. In the other problems not solved at optimality, computation halted due to memory limit.

- Percentage gaps observed between the optimum (or the best lower bound) of CS-LOSP and that of CSP (again, always equal to $\lceil z^R_{CSP} \rceil$) are: 1.39% for $s = 2$ (0.03% for $s = 3$) on average, 3.7% (0.3%) maximum, and no gap in 3 (18) cases.

- The lower bound computed by track enumeration is always identical to $z^R_{CSP(s)}$.

- In problems with $s = 2$ ($s = 3$), the average gap improved from 1.11% (0.43%) to 0.23% (0.28%). For problems not solved to optimality, the heuristic solution exceeds the lower bound by ≤ 2 (≤ 3) stock items.

- Optimal solutions, when found, required 248.97 seconds on average. In all the problems (but the first) for which an optimal solution could not be found, CPLEX reached memory limits quite rapidly. In fact, the models solved have a non-negligible amount of variables: the mean number of feasible patterns is 326 for $s = 2$ and 2002 for $s = 3$.

6 Conclusions and future research

We consider a problem of stock cutting where cuts must be sequenced so that the number of distinct part types produced at any time (open stacks) does not exceed a prescribed s. The problem is discussed referring to recent literature on cutting and sequencing problems, and formulated as integer linear programming. Like the formulation given in [17], the one proposed has exponentially many variables, but

unlike it, it has much less constraints (in fact, quadratically many in the number of part types). A preliminary computational test performed on a set of small instances indicates that the model (i) is computationally viable for achieving good feasible and even optimal solution of non-trivial instances with up to 20 part types, and (ii) is much more manageable — also in terms of convergence — than existing methods. Further research must however be done in order to design an exact algorithm for large instances.

References

[1] Arbib, C., Marinelli, F., Pezzella, F. (2012) An LP-based tabu search for batch scheduling in a cutting process with finite buffers. International Journal of Production Economics, 136(2), 287-296.

[2] Arbib, C., Marinelli, F. (2009) Exact and Asymptotically Exact Solutions for a Class of Assortment Problems. INFORMS Journal on Computing, 21(1), 13-25.

[3] Arbib, C., Di Iorio, F., Marinelli, F., Rossi, F. (2002) Cutting and Reuse: an Application from Automobile Component Manufacturing. Operations Research, 50(6), 923-934.

[4] Belov, G., Scheithauer, G. (2007) Setup and Open-Stacks Minimization in One-Dimensional Stock Cutting. INFORMS Journal on Computing. 19(1), 27-35.

[5] Ben Amor, H., Desrosiers, J., Frangioni, A. (2004) Stabilization in column generation. Les Cahiers du GERAD, tech. rep. G-2004-62.

[6] Chu, G. G., Stuckey, P. (2009) Minimizing the Maximum Number of Open Stacks by Customer Search. *Lecture Notes in Computer Science: Principles and Practice of Constraint Programming CP2009*, Springer-Verlag, Vol. 5202, 242-257.

[7] De Giovanni, L., Pezzella, F., Pfetsch, M., Rinaldi, G., Ventura, P. (2007) The Open Stack Problem. *Giornate di Lavoro AIRO*, Genova, Italy.

[8] Dyckhoff, H., Scheithauer, G., Terno, J. (1996) Cutting and Packing: An Annotated Bibliography. `ibl@ibl.RWTH-aachen.de`.

[9] Gilmore, P.C, Gomory, R.E. (1961) A Linear Programming Approach to the Cutting Stock Problem. Operations Research, 8, 849-859.

[10] Gilmore, P.C., Gomory, R.E. (1963) A Linear Programming Approach to the Cutting Stock Problem – Part II. Operations Research, 11, 863-888.

[11] Kantorovich, L.V. (1960) Mathematical Models of Organising and Planning Production. Management Science, 6, 366-422.

[12] Lübbecke, M., Desrosiers, J., (2004) Selected Topics in Column Generation. Operations Research, 53, 1007-1023.

[13] Matsumoto, K., S. Umetani, H. Nagamochi (2011) On the one-dimensional stock cutting problem in the paper tube industry. Journal of Scheduling, 14(3), 281-290.

[14] Muter, I., Ilker Birbil, S., Bülbül, K. (2012) Simultaneous column-and-row generation for large-scale linear programs with column-dependent-rows. Mathematical Programming A, 1-36.

[15] Smith, B.M., Gent, I.P. (2005) Constraint Modelling Challenge 2005. *Fifth Workshop on Modelling and Solving Problems with Constraints*, IJCAI 2005, Edinburgh, Scotland.

[16] Tang, C.S., Denardo, E.V. (1988) Models Arising from a Flexible Manufacturing Machine, Part I: Minimization of the Number of Tool Switches. Operations Research, 36, 767-777.

[17] Yanasse, H.H., Pinto Lamosa, M.J. (2007) An integrated cutting stock and sequencing problem. European Journal of Operational Research, 183, 1353-1370.

Table 1: Results for the instances with 10 part types (only with $z_{UB} - z^R_{CSP(s)} \geq 1$).

					track_enum()			model (13)-(20)		
ID	s	$\|K\|$	z^R_{CSP}	$z^R_{CSP(s)}$	z_{LB}	calls	time (sec.)	z^*	time (sec.)	z_{UB}
1	2	131	85.90	90.76	90.83	253	0.36	91	7.77	92
4	2	113	121.92	125.13	125.86	52	0.24	126	0.75	127
5	2	164	96.82	97.65	97.84	280	0.30	98	100.78	99
6	2	163	99.25	99.84	100.00	284	0.30	101	64.53	101
7	2	175	121.13	121.54	121.96	28	0.25	122	3.94	123
9	2	136	107.99	108.95	108.98	186	0.23	110	3.30	111
12	2	168	96.11	96.87	97.00	604	0.38	98	17.41	99
13	2	124	143.40	145.21	145.95	177	0.22	146	2.44	147
14	2	182	100.67	101.97	102.01	2120	0.84	103	13.09	103
15	2	164	86.48	86.61	86.95	47	0.23	87	17.91	88
19	2	251	114.76	116.51	117.00	139	0.36	117	270.61	118
20	2	141	96.14	97.55	97.99	4618	1.33	98	60.94	99
22	2	107	119.37	119.83	120.00	163	0.22	120	0.89	121
24	2	121	141.25	145.83	145.89	421	0.30	146	21.83	147
28	2	110	120.00	120.00	120.00	673	0.34	121	1800.00	122
29	2	114	181.01	185.56	185.99	243	0.25	186	66.78	187
30	2	116	109.50	110.56	110.94	295	0.27	111	1.47	112
34	2	109	114.81	116.89	116.89	274	0.27	118	131.02	118
35	2	121	115.89	117.58	117.58	380	0.30	118	23.36	119
1	3	599	85.90	86.83	87.29	12823	11.38	88	69.83	88
2	3	655	112.05	112.45	112.94	443	1.14	113	4.56	114
4	3	362	121.92	121.98	122.14	3455	2.34	123	6.33	123
5	3	846	96.82	96.87	96.99	244	1.22	97	403.74	98
9	3	641	107.99	107.99	107.99	768	1.45	109	1005.64	109
11	3	1824	76.95	76.96	77.00	6516	17.74	78	1564.53	78
13	3	433	143.40	143.49	143.94	3077	2.56	144	25.91	145
14	3	1070	100.67	100.85	100.98	4594	7.92	101	500.03	102
23	3	290	118.61	118.63	118.85	1751	1.17	119	3.63	120
24	3	381	141.25	141.25	141.45	3791	2.70	142	3.19	143
25	3	293	166.63	166.63	166.99	11422	5.70	168	90.34	168
26	3	292	164.28	164.92	164.92	1050	0.86	165	1.84	166
32	3	336	135.56	136.97	136.97	3224	2.09	137	5.72	138
37	3	295	138.39	138.95	139.03	733	0.75	140	21.94	140

Table 2: Results for instances with 20 part types.

					model (13)-(20)			
ID	s	$\|K\|$	z^R_{CSP}	$z^R_{CSP(s)}$	z_{UB}	time (sec.)	z_{LB}	z_{UB}
1	2	237	239.41	243.93	245	3601.0	243.97	246
2	2	287	406.08	408.96	411	956.7	409.00	411
3	2	292	350.96	363.61	**364**	1444.8	364.00	369
4	2	224	412.00	413.08	**414**	69.8	414.00	414
5	2	328	308.47	311.10	313	636.7	311.10	313
6	2	317	392.08	399.71	402	655.8	399.71	405
7	2	344	337.00	341.25	**342**	8.9	342.00	342
8	2	308	425.00	425.00	**425**	2.0	425.00	425
9	2	268	340.29	347.77	349	1849.1	347.81	359
10	2	421	363.00	373.30	375	1064.8	373.33	382
11	2	544	236.14	239.24	242	503.0	239.24	248
12	2	290	436.50	436.50	**437**	26.1	437.00	439
13	2	257	342.08	347.88	**349**	651.7	349.00	352
14	2	362	350.62	355.53	357	886.8	355.53	359
15	2	287	373.00	377.76	379	1879.6	377.88	380
16	2	245	321.64	323.60	**324**	1003.2	324.00	325
17	2	390	355.00	355.00	**355**	7.6	355.00	368
18	2	464	266.03	269.46	271	1160.3	269.46	274
19	2	387	329.69	334.08	336	554.5	334.08	337
20	2	275	385.46	397.54	399	1093.3	397.54	399
1	3	1010	239.41	239.41	**240**	159.9	240.00	241
2	3	1296	406.08	406.08	408	983.0	406.08	408
3	3	1385	350.96	351.94	355	889.9	351.94	355
4	3	623	412.00	412.00	**412**	16.8	412.00	412
5	3	1652	308.47	309.00	**310**	34.7	310.00	310
6	3	1456	392.08	392.08	395	755.4	392.08	395
7	3	2134	337.00	337.00	**337**	260.0	337.00	338
8	3	1752	425.00	425.00	**425**	6.0	425.00	425
9	3	1200	340.29	340.29	**341**	307.0	341.00	343
10	3	3833	363.00	363.00	366	2477.5	363.00	368
11	3	4784	236.14	236.34	239	1037.8	236.34	238
12	3	1625	436.50	436.50	**437**	11.6	437.00	437
13	3	840	342.08	342.08	**343**	154.1	343.00	344
14	3	2092	350.62	350.72	353	930.4	350.72	352
15	3	1478	373.00	373.00	374	1836.3	373.00	375
16	3	806	321.64	321.64	**322**	268.8	322.00	323
17	3	3302	355.00	355.00	**355**	48.4	355.00	356
18	3	4537	266.03	266.11	269	1628.2	266.11	270
19	3	3067	329.69	329.69	331	1147.4	329.69	331
20	3	1174	385.46	385.46	388	663.6	385.46	388